

Article

HyperCross: A Semantic-Aware Zero-Knowledge Indexing Framework for Cross-Chain Data

Kun Hao ^{1,*}  and Yuliang Ma ²¹ School of Software, Shenyang University of Technology, Shenyang 110870, China² College of Information Science and Engineering, Northeastern University, Shenyang 110819, China; mayuliang@mail.neu.edu.cn

* Correspondence: haokun@sut.edu.cn

Abstract

The transition from isolated distributed ledgers to a unified “Internet of Value” is hindered by the lack of efficient, verifiable, and privacy-preserving cross-chain data retrieval mechanisms. While asset bridging has matured, generalized data indexing remains a critical bottleneck, constrained by the semantic gap between heterogeneous storage layouts and the prohibitive verification tax of cryptographic proofs. In this paper, we present HyperCross, a novel semantic-aware zero-knowledge indexing framework designed to bridge this divide. We first formalize the heterogeneous cross-chain storage optimization problem (HCCSOP) and prove its NP-completeness. To tackle this, HyperCross employs a synergistic tri-layered architecture. At the semantic layer, we introduce a unified data abstraction (UDA) that leverages category-theoretic functors and schema morphisms to ensure mathematically rigorous state mapping for both simple assets and complex smart contract logic. At the indexing layer, a zero-knowledge learning index (ZKLI) shifts prediction intelligence to the client side, integrating zk-SNARKs with silent oblivious transfer to achieve constant-time verification ($O(1)$) while concealing access patterns. Finally, a multi-level cache (MLC) utilizes predictive prefetching with Δ -bounded staleness to mask network latency. Extensive evaluations demonstrate that HyperCross reduces query latency by $2.4\times$ and storage overhead by 40% compared to state-of-the-art baselines, establishing a scalable foundation for data-intensive inter-chain applications.

Keywords: cross-chain; data indexing; zero-knowledge proof; semantic-aware; storage optimization



Academic Editor: Hung-Yu Chien

Received: 13 March 2026

Revised: 8 April 2026

Accepted: 16 April 2026

Published: 20 April 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

The blockchain ecosystem is undergoing a fundamental transformation from isolated silos into an interconnected “Internet of Value,” where decentralized applications (dApps) increasingly operate across multiple networks. From DeFi aggregators optimizing yields across chains to cross-chain governance and decentralized identity (DID) systems, the capability to access and utilize data from diverse ledgers is becoming a prerequisite. However, while significant progress has been made in *asset* interoperability (e.g., token bridges, atomic swaps) [1,2], the problem of efficient, secure, and privacy-preserving cross-chain *data* retrieval remains largely unresolved.

Cross-chain data retrieval introduces a distinct set of challenges that current solutions fail to address comprehensively. Primarily, the **semantic gap** between ledger models—specifically the unspent transaction output (UTXO) model of Bitcoin and the Account/Balance model of Ethereum—renders direct data querying error-prone. Without a rigorous

formalism, ad-hoc conversions can violate state invariants, precipitating security vulnerabilities. Concurrently, the **verification tax** is prohibitive. Verifying a remote state typically necessitates running a light client or checking Merkle proofs on-chain, which incurs high gas costs and latency. Furthermore, the **privacy–efficiency trade-off** is stark. Plaintext indexing offers high performance but leaks access patterns, whereas private information retrieval (PIR) or encrypted search schemes often introduce unacceptable latency or storage overheads [3,4].

Existing solutions often compromise on one or more of these dimensions. Centralized indexers (e.g., The Graph) offer speed but reintroduce trust assumptions. Light clients provide security but are difficult to scale for data-intensive queries. Cross-chain bridges focus on asset locking and minting, lacking the granularity for general-purpose data queries. In the verifiable-query line, systems such as vSQL [5] and vChain [6] provide important building blocks but are not designed for heterogeneous cross-chain semantics. In the indexing line, learned indexes [7,8] reduce storage and lookup cost, yet they are typically not paired with zero-knowledge verification for adversarial cross-chain workloads. In the interoperability line, protocol frameworks such as Polkadot/Cosmos [9,10] standardize message passing but treat payload semantics as external concerns.

To mitigate these limitations, we propose *HyperCross*, a holistic storage and indexing framework designed to bridge the gap between heterogeneous blockchains. HyperCross treats the cross-chain data management problem as a multi-objective optimization challenge, balancing semantic fidelity, query performance, and privacy. We articulate these core design challenges as the *heterogeneous cross-chain storage optimization problem (HCCSOP)*, which we formalize and prove to be NP-complete.

This theoretical grounding motivates our heuristic-based, three-layered architecture that systematically resolves the tension between these objectives. First, we tackle the semantic gap through **semantic unification via category theory**. The unified data abstraction (UDA) layer employs category-theoretic functors and natural isomorphisms to map diverse ledger states (e.g., UTXO, Account) into a canonical format, ensuring that data transformations are mathematically sound and invariant-preserving. Second, to address the privacy–efficiency trade-off, we introduce **verifiable privacy with ZK-learned indexes**. The zero-knowledge learning index (ZKLI) fuses adaptive learned indexes with zero-knowledge proofs (specifically zk-SNARKs), allowing a querier to prove the correctness of a query result—both membership and range—without revealing the specific key being accessed or the underlying dataset distribution. This construction achieves constant-time on-chain verification while preserving strict privacy. Finally, to mitigate the verification tax, we ensure **performance via predictive caching**. The multi-level cache (MLC) system employs predictive placement and ensures Δ -bounded staleness, bridging the latency gap between on-chain storage and application needs by balancing data freshness with high-speed responsiveness. Our novelty is therefore at the level of cross-layer composition: HyperCross unifies semantic mapping, verifiable private indexing, and cache-aware execution into one deployable cross-chain retrieval pipeline, while each underlying primitive remains grounded in established prior work.

This paper makes the following contributions:

- **Formal Foundation:** We formally define the HCCSOP and establish its computational hardness (NP-completeness), providing a rigorous basis for system optimization.
- **Novel Architecture:** We present HyperCross, a unified framework that integrates category-theoretic semantic unification with zero-knowledge learned indexes to jointly optimize interoperability, privacy, and efficiency.

- **Zero-Knowledge Indexing:** We detail the design of ZKLI, demonstrating how to generate succinct proofs for learned index lookups, a technique with broader applications beyond cross-chain storage.
- **Implementation and Evaluation:** We implement a prototype of HyperCross and evaluate it against state-of-the-art baselines. Experimental results show that HyperCross reduces query latency by significant margins while maintaining a compact storage footprint and robust privacy guarantees.

The remainder of this paper is organized as follows. Section 2 formalizes the HCCSOP. Section 3 provides an architectural overview. Sections 4–6 detail UDA, ZKLI, and MLC, respectively. Section 7 presents system analysis covering correctness, consistency, security, and reliability. Section 8 reports experimental results. Section 9 reviews related work. Section 10 discusses scope and limitations. Section 11 concludes the paper.

2. Problem Definition

In this section, we formally define the heterogeneous cross-chain storage optimization problem (HCCSOP). Our goal is to develop a unified storage engine that provides efficient, secure, and consistent access to data distributed across multiple heterogeneous blockchain networks.

Let $\mathcal{N}$ denote the set of blockchain networks. Let $\mathcal{C}_U$ and $\mathcal{C}_A$ be the categories that capture states and valid transitions for UTXO and Account models, respectively. A query q operates over keys k with commitments $C(k)$, predicted positions p_{pred} , window size ϵ , and Merkle root R_{Merkle} .

Let $\mathcal{N} = \{N_1, N_2, \dots, N_k\}$ be a set of k distinct blockchain networks. Each network N_i has its own transaction model $M_i \in \{\text{UTXO}, \text{Account}, \text{Hybrid}\}$, data schema S_i , consensus protocol C_i , and a set of stored data objects D_i . A query q is a request for a set of data objects that may span multiple networks.

Definition 1 (HCCSOP). *Given the set of networks $\mathcal{N}$, a workload of queries $Q = \{q_1, q_2, \dots, q_m\}$, and a set of performance and security requirements, the HCCSOP is to design a storage system that minimizes a cost function $\mathcal{F}_{cost}$ while satisfying a set of constraints.*

$$\min \mathcal{F}_{cost} = w_1 \cdot L_{avg} + w_2 \cdot S_{total} + w_3 \cdot C_{comm} \quad (1)$$

where L_{avg} is the average query latency, S_{total} is the total storage footprint, and C_{comm} is the cross-chain communication overhead. The weights w_i represent the relative importance of each objective.

Definition 2 (HCCSOP-Decision). *Given budgets W and a bound $B \in \mathbb{R}_{\geq 0}$, decide whether there exists a configuration that satisfies all constraints and whose cost $\mathcal{F}_{cost}$ is at most B .*

To render the optimization well posed and to bound the feasible set for both analysis and implementation, we adopt three working assumptions that delimit admissible configurations. These assumptions concern semantic preservation across ledger models, privacy guarantees of the indexing and proof mechanisms, and storage consistency under Δ -bounded staleness. They do not alter the objective; rather, they restrict the search space and make trade-offs measurable, providing the basis for the hardness analysis and architectural design that follow. The key symbols used in this section are summarized in Table 1.

Table 1. Notation summary.

Symbol	Description
$\mathcal{N}$	Set of blockchain networks
N_i	The i -th network
M_i	Transaction model of N_i (UTXO/Account/Hybrid)
S_i	Data schema of N_i
C_i	Consensus protocol of N_i
D_i	Data objects stored on N_i
q	Query over data objects
Q	Workload of queries
k	Query key
$C(k)$	Commitment of key k
p_{pred}	Predicted position for k
ϵ	Window size for verification
R_{Merkle}	Merkle root of committed dataset
$\mathcal{F}_{cost}$	System cost objective
w_1, w_2, w_3	Objective weights
L_{avg}	Average query latency
S_{total}	Total storage footprint
C_{comm}	Cross-chain communication cost
$v_{auth}(k, t)$	Authoritative version at time t
$v_{cache}(k, t)$	Cached version at time t
Δ	Staleness bound
$age(k, t)$	Version age at time t
$\mathcal{C}_U, \mathcal{C}_A$	Categories for UTXO/Account models
w_i	Size vector (MKP/HCCSOP mapping)
v_i	Benefit/value of item i
W	Capacity constraints
V	Target value threshold
o_i	Data object corresponding to item i
d	Number of resource dimensions

Assumption 1 (Semantic Preservation). Let $\mathcal{C}_i$ and $\mathcal{C}_j$ be the categories that capture the states and valid transitions of transaction models M_i and M_j . There exist functors $F_{ij} : \mathcal{C}_i \rightarrow \mathcal{C}_j$ and $F_{ji} : \mathcal{C}_j \rightarrow \mathcal{C}_i$ together with natural isomorphisms $\eta : \text{Id}_{\mathcal{C}_i} \Rightarrow F_{ji} \circ F_{ij}$ and $\epsilon : \text{Id}_{\mathcal{C}_j} \Rightarrow F_{ij} \circ F_{ji}$ that preserve transaction composition and invariants. Thus, semantic conversion is lossless up to natural isomorphism.

Assumption 2 (Privacy Guarantee). For any polynomial-length query sequence Q over keys, there exists a probabilistic polynomial-time simulator S such that the server's view $\text{View}_A(Q)$ in the ZKLI protocol and the simulated view $\text{Sim}_S(Q)$ are computationally indistinguishable. Leakage is restricted to public metadata (e.g., transcript length, proof size) and does not include the access pattern to specific keys.

Assumption 3 (Storage Consistency). Let $v_{auth}(k, t)$ be the authoritative version of key k at time t on its source chain and $v_{cache}(k, t)$ be the version served by MLC. There exists a bound $\Delta > 0$ such that for all k and t , the staleness $age(k, t) = \text{time}(v_{auth}(k, t)) - \text{time}(v_{cache}(k, t)) \leq \Delta$, with read-your-writes preserved per client session. When $age(k, t) > \Delta$, the cache invalidates and refreshes before serving.

Theorem 1. The decision version of HCCSOP (HCCSOP-Decision) is NP-complete.

Proof. We reduce the multidimensional knapsack problem (MKP-Decision) to HCCSOP-Decision. An MKP instance has items with size vector $w_i \in \mathbb{R}_{\geq 0}^d$, value v_i , knapsack capacities $W \in \mathbb{R}_{\geq 0}^d$, and target V . Construct an HCCSOP instance with a data object o_i per item, a binary selection variable $x_i \in \{0, 1\}$, and d capacity constraints matching W .

Consider the special case where $\mathcal{F}_{cost} = L_{avg}$ with weights $(w_1, w_2, w_3) = (1, 0, 0)$, and fix other constraints to hold trivially. Let L_{base} be a constant baseline latency and define $L_{avg}(X) = L_{base} - \sum_i v_i x_i$. Set the decision bound $B = L_{base} - V$. Then, there exists a feasible MKP solution with $\sum_i v_i x_i \geq V$ and $\sum_i w_i x_i \leq W$ if and only if there exists a configuration for HCCSOP with $\mathcal{F}_{cost}(X) \leq B$ and capacities satisfied. The construction is polynomial and preserves feasibility; hence, HCCSOP-Decision is NP-hard. Membership in NP holds because, given a configuration X , we can evaluate $\mathcal{F}_{cost}(X)$ and check all constraints in time polynomial in the input size. Therefore, HCCSOP-Decision is NP-complete. $\square$

Given the computational intractability of the optimal solution, we proceed to design HyperCross as a heuristic-based architectural framework. Specifically, HyperCross approximates the optimal solution by prioritizing latency minimization (w_1) via the MLC layer and communication efficiency (w_3) via the ZKLI layer, while satisfying the semantic and privacy constraints through UDA and OT, respectively.

Practical Mapping from HCCSOP to System Knobs

To clarify the practical role of HCCSOP, Table 2 maps formal decision variables and constraints to concrete engineering knobs used in our implementation.

Table 2. Mapping of HCCSOP variables/constraints to HyperCross implementation decisions.

Formal Term	Operational Meaning	HyperCross Knob/Enforcement
$w_1 \cdot L_{avg}$	Latency objective	Cache tiering policy, predictor choice, prefetch depth k , verification mode (eager/lazy)
$w_2 \cdot S_{total}$	Storage objective	Learned model type (piecewise linear), model package size, Merkle depth, cache capacity $ C_1 , C_2 $
$w_3 \cdot C_{comm}$	Communication objective	OT window size, cache hit targets, cross-chain batching, invalidation granularity
Semantic constraints	Invariant-preserving conversion	UDA functors F_{UA}, F_{AU} and round-trip consistency checks
Privacy constraints	Access-pattern and witness secrecy	Commitment + OT transcript indistinguishability + zk-SNARK witness hiding
Consistency constraints	Freshness bound	Δ -bounded staleness and reorg-triggered invalidation in MLC

In practice, system tuning proceeds as follows: (i) choose (w_1, w_2, w_3) according to service-level objectives; (ii) derive cache and index budgets from S_{total} ; (iii) calibrate OT window and prefetch depth to satisfy communication and latency budgets; and (iv) validate constraints through semantic round-trip tests and Δ -consistency checks. This mapping makes the NP-hard formulation actionable as a configuration blueprint rather than a purely theoretical result.

3. Overview

HyperCross is a hierarchical cross-chain storage engine designed to systematically address the challenges of semantic heterogeneity, privacy-preserving indexing, and practical performance. Its architecture, illustrated in Figure 1, unifies three core layers into a cohesive retrieval pipeline.

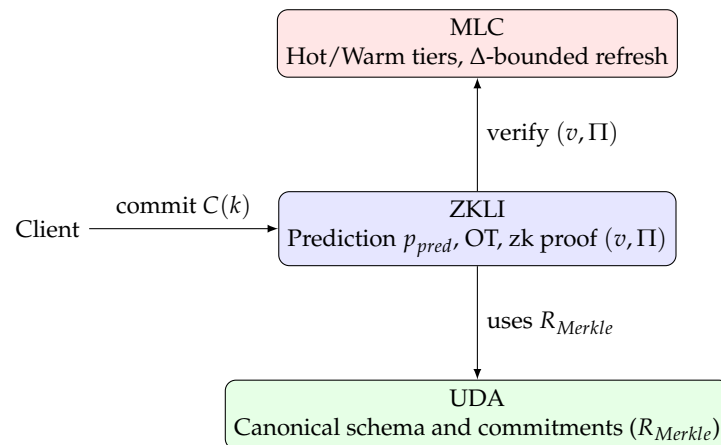


Figure 1. HyperCross architecture with three layers: UDA (semantics), ZKLI (privacy and verifiability), and MLC (performance and consistency).

At the foundation, the **unified data abstraction (UDA) layer** provides semantic reconciliation via functorial equivalence and natural isomorphisms between ledger models. Rather than relying on ad-hoc translation scripts, UDA defines a canonical object and operation space so that queries and conversions are invariant-preserving and deterministic across UTXO, Account, and hybrid chains. It exposes interfaces for normalization and conversion (e.g., schema harmonization and transaction mapping) and produces dataset commitments (e.g., Merkle roots) consumed by upper layers. Ledger-specific metadata are folded into canonical summaries to avoid semantic drift while retaining sufficient provenance for verification.

Building upon the semantic foundation, the **zero-knowledge learning index (ZKLI) layer** constructs a verifiable, privacy-preserving index by coupling adaptive learned prediction with zk proofs. For a committed key, ZKLI predicts a position and verifies membership within a bounded window using Merkle circuits optimized for modern zk systems and field-friendly hashes. Crucially, it employs oblivious transfer to hide item selection within the window, restricting leakage to public transcript features. This design achieves sub-logarithmic average-case query cost with worst-case logarithmic verification, supports bounded-range checks with indistinguishable window expansion on miss, and provides a uniform verification primitive for downstream use.

To bridge the latency gap between on-chain storage and application needs, the **multi-level cache (MLC) layer** sits at the top of the stack. It implements hot and warm tiers backed by source chains, combining recency/frequency-aware replacement with a lightweight predictor for prefetch. MLC gates responses through proof verification and enforces Δ -bounded staleness with refresh or invalidation on demand. By supporting both verification-first and performance-first modes, and instrumenting tier hit rates and latency breakdowns, MLC makes consistency and performance trade-offs observable and manageable for the end user.

4. Unified Data Abstraction

The primary impediment to seamless cross-chain data retrieval is the *semantic gap* existing between heterogeneous ledger models. While basic asset transfers distinguish between unspent transaction output (UTXO) and Account/Balance models, modern decentralized applications (dApps) introduce a more profound challenge: *generalized state heterogeneity*. Smart contracts on different chains employ divergent storage layouts (e.g., Solidity's slot-based storage vs. WASM-based linear memory) and logic representations. Direct queries across these disparate models are inherently error-prone; a valid state transition in one paradigm does not naturally map to the other without violating critical invariants.

Ad-hoc translation scripts frequently fail to capture these subtle semantic nuances, leading to “semantic drift.” To circumvent this, we require a rigorous mathematical framework capable of guaranteeing lossless conversion and invariant preservation for both simple assets and complex contract states.

4.1. Category Theoretic Unification

To bridge this semantic schism, the unified data abstraction (UDA) layer leverages category theory to formalize the relationships between ledger states. We model the diverse blockchain state spaces (UTXO, Account, contract storage) as distinct categories, $\mathcal{C}_{Src}$ and $\mathcal{C}_{Tgt}$. We define the conversion process as a *functorial map* $\mathcal{F} : \mathcal{C}_{Src} \rightarrow \mathcal{C}_{Tgt}$.

4.1.1. Generalized State Objects

We define a canonical state object O in the target category $\mathcal{C}_{Tgt}$ as a tuple $O = \langle ID, \Sigma, \Pi \rangle$, where ID is a globally unique identifier (e.g., derived from contract address and storage key), Σ represents the semantic value (e.g., balance, liquidity parameter), and Π denotes provenance metadata (e.g., block height, proof context).

4.1.2. Functorial State Mapping

The functor $\mathcal{F}$ establishes a structural correspondence between source and target categories:

- **Asset Morphism:** For native assets, $\mathcal{F}_{Asset}$ maps sets of UTXOs to aggregated Account objects (and vice versa), ensuring value conservation ($\sum v_{in} = \sum v_{out}$).
- **Schema Morphism:** For smart contracts, $\mathcal{F}_{Schema}$ maps platform-specific storage layouts (e.g., EVM slots) to canonical typed objects. This allows, for instance, a Uniswap V3 position on Ethereum to be queried as a generic ‘LiquidityPosition’ object on a Polkadot parachain, abstracting away the underlying slot calculation details.

Crucially, we establish a *natural isomorphism* $\eta : \text{Id}_{\mathcal{C}_{Src}} \rightarrow \mathcal{F}^{-1} \circ \mathcal{F}$. This ensures that the conversion is reversible and lossless: converting a complex state to its canonical form and back yields a state isomorphic to the original.

Example 1 (Dual-Mode Mapping). *Case A: UTXO to Account.* A set of Bitcoin UTXOs $\{(tx_1, 0, 0.5), (tx_1, 1, 1.5)\}$ is mapped via $\mathcal{F}_{Asset}$ to $\{Addr : H(PK), Bal : 2.0\}$. *Case B: Contract State.* An Ethereum ERC-20 contract storage at ‘Slot[MapHash(User, 0)]’ is mapped via $\mathcal{F}_{Schema}$ to a canonical object $\{Type : TokenBalance, Owner : User, Value : uint256(Slot)\}$. This decouples the data from the EVM’s specific storage layout.

4.2. Algorithm Description

The canonical conversion process is implemented as a deterministic, stateless transformation, as delineated in Algorithm 1. The algorithm is designed to be extensible: while the core logic handles asset aggregation, the ApplyFunctor procedure can be extended with plugin-based *Schema Adaptors* to handle specific contract standards (e.g., ERC-20, ERC-721).

4.3. Complexity Analysis

The computational complexity of the UDA conversion is dominated by the aggregation and sorting steps. Let N be the number of data items (UTXOs or Accounts) in the source state S .

- **Aggregation ($U \rightarrow A$):** The GroupBy operation typically requires a hash map or sorting, taking $O(N)$ or $O(N \log N)$ time, respectively. Iterating through groups to sum balances is linear with respect to the number of inputs, $O(N)$.
- **Sharding ($A \rightarrow U$):** Generating UTXOs from accounts is a direct linear map, $O(N)$.

- **Canonical Sorting:** Sorting the output state S' is necessary for Merkle determinism and takes $O(N \log N)$.

Algorithm 1 Generalized Canonical Functorial Conversion

Require: Source state S , Schema Type $T \in \{\text{Asset}, \text{Contract}\}$

Ensure: Canonical Target State S'

```

1: procedure APPLYFUNCTOR( $S, T$ )
2:    $S' \leftarrow \emptyset$ 
3:   if  $T = \text{Asset}$  then
4:      $\triangleright$  Aggregate UTXOs or Shard Accounts (Standard Logic)
5:      $S' \leftarrow \text{ProcessAssetMapping}(S)$ 
6:   else if  $T = \text{Contract}$  then
7:      $\triangleright$  Apply Schema Morphism for Complex Logic
8:     for all  $(key, val) \in S$  do
9:        $schema \leftarrow \text{LoadSchema}(S.\text{contract\_id})$ 
10:       $obj \leftarrow \text{LiftToCanonical}(key, val, schema)$ 
11:       $S' \leftarrow S' \cup \{obj\}$ 
12:     end for
13:   end if
14:   return CanonicalSort( $S'$ )  $\triangleright$  Ensure Merkle Root Determinism
15: end procedure
  
```

Therefore, the overall time complexity of the UDA layer is $O(N \log N)$. Since this transformation is stateless and parallelizable across partitions, it scales efficiently for large blockchain states and does not become a bottleneck in the retrieval pipeline. Figure 2 illustrates the conversion workflow.

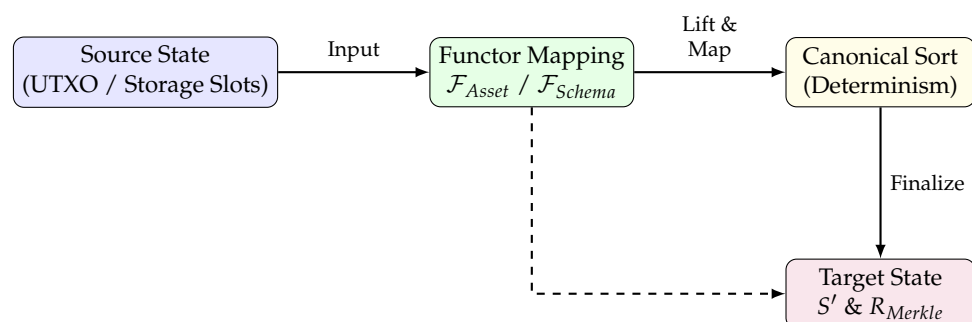


Figure 2. Workflow of the generalized canonical functorial conversion (UDA). Heterogeneous states (assets or logic) are transformed via specialized functors.

5. Zero-Knowledge Learning Index

Although the UDA layer resolves semantic inconsistencies, it does not address the *privacy–efficiency trade-off* inherent in cross-chain indexing. Traditional authenticated structures like Merkle trees provide integrity but suffer from two major limitations. Primarily, standard Merkle proofs (a path of hashes) grow logarithmically with dataset size, leading to expensive on-chain verification costs (“gas”). Subsequently, and more critically, submitting a proof reveals the traversal path, which leaks the specific key being accessed to the verifier (the destination chain). Conversely, private retrieval schemes often incur prohibitive latency. To resolve this dilemma, we propose the zero-knowledge learning index (ZKLI), which unifies the constant-time lookup performance of learned indexes with the privacy guarantees of zero-knowledge proofs (ZK-SNARKs).

5.1. Verifiable Learned Indexing

The ZKLI architecture reconceptualizes the indexing responsibility by shifting the *prediction intelligence* to the client side while keeping the *data storage* on the server. This separation of concerns allows us to decouple the query intent from the data retrieval mechanism.

5.1.1. Client-Side Learned Model

Unlike traditional database indexes where the B-tree or hash table resides entirely on the server, ZKLI trains a compact learned model $M : \mathcal{K} \rightarrow \mathcal{P}$ over the dataset's keys. This model approximates the cumulative distribution function (CDF) of the sorted keys. Crucially, the model parameters (e.g., weights of a linear regression or a small neural network) are lightweight (KB-sized) and are distributed to clients. When a client wishes to query a key k , it executes $M(k)$ locally to obtain a predicted physical position p_{pred} . This local computation is the cornerstone of our privacy guarantee: since the server never sees the key k during the lookup phase, it cannot infer the access pattern from the lookup logic itself.

5.1.2. Formal Circuit Definition

To bridge the “trust gap” between the client and the source chain, we define a ZK-SNARK circuit C_{index} that implements a verifiable lookup logic. This circuit enables the generation of a succinct proof Π attesting that a retrieved raw value v_{raw} is indeed the authoritative state associated with a key k committed in the global state root R .

Definition 3 (ZKLI Circuit Relation). Let $\mathbb{F}_p$ be the scalar field of the BN254 elliptic curve. We define the ZKLI circuit relation $\mathcal{R}_{ZKLI}$ as a set of tuples (x, w) , where x is the public instance and w is the private witness. The relation holds, i.e., $\mathcal{R}_{ZKLI}(x, w) = 1$, if and only if the constraints listed below are satisfied.

Instance $x = (C, R)$:

- $C \in \mathbb{F}_p$: The cryptographic commitment to the query key k .
- $R \in \mathbb{F}_p$: The trusted Merkle root of the source chain state.

Witness $w = (k, v_{raw}, r, \mathcal{P}, \mathcal{H})$:

- $k \in \mathbb{F}_p$: The search key (e.g., account address or storage slot).
- $v_{raw} \in \mathbb{F}_p$: The raw value stored on-chain (input to UDA).
- $r \in \mathbb{F}_p$: The randomness for the commitment C .
- $\mathcal{P}$: The Merkle inclusion proof.
- $\mathcal{H}$: The hashing trace.

5.1.3. Implementation Profile and Reproducibility Parameters

Our implementation instantiates ZKLI with **Groth16** over **BN254** using the gnark backend, with MiMC as the in-circuit hash. The Merkle path depth is fixed to 20 (`MerkleTreeDepth = 20`) to support million-scale state indexing. A direct compilation of the deployed circuit shape reports **14,582 constraints** and public/secret input counts of 2/43, respectively.

To expose the operational cost, we report benchmarked values from the released implementation: (i) end-to-end query with proof generation: **126.9 ms/op**; (ii) proof verification: **0.538 ms/op**. These numbers are obtained on an Apple M4 under go test benchmarks and are included to make proof-system overhead explicit for reproducibility.

For OT integration, ZKLI uses the predicted position to define a bounded retrieval window and executes oblivious transfer over encrypted candidates inside that window. The verifier receives only the public instance (commitment and Merkle root) and proof transcript; no raw key or witness values are disclosed.

Smart-contract verifier interface. To make on-chain integration explicit, HyperCross exposes a minimal verifier-facing interface with three operations: (1) `submitRoot(chainId, root, epoch)` to register finalized source-chain commitments; (2) `verifyQuery(commitment, root, proof, publicSignals)` to validate Groth16 proofs against the active verifier key; and (3) `resolveValue(objectId, canonicalValue, proofRef)` to bind verified outputs to application-level state transitions. The contract-side logic remains lightweight by checking only proof validity and root consistency, while heavy retrieval/prediction remains off-chain. This separation keeps gas bounded and preserves the trust model described in Section 7.

While UDA ensures semantic consistency and ZKLI provides privacy-preserving verification, a critical practical challenge remains: the *performance–latency gap*. On-chain data retrieval is inherently slow due to network consensus latency (seconds to minutes) and the overhead of cryptographic proof generation. Modern dApps, however, require millisecond-level responsiveness. Direct querying of the source chain for every request is infeasible. Traditional caching solutions fail in this context because they lack *verifiable consistency*—a cached item might be stale or tampered with, and there is no standard mechanism to verify its integrity against the current on-chain state without re-fetching the data. Thus, the challenge is to design a caching layer that bridges this latency gap while strictly enforcing the freshness and integrity guarantees provided by the underlying ZK-SNARKs.

6. Multi-Level Cache

6.1. Predictive Hierarchical Caching

To circumvent this bottleneck, the MLC layer implements a hierarchical storage architecture governed by a predictive pre-fetching policy. As shown in Figure 3, the system is organized into three tiers:

- **Hot Cache (C_1):** An in-memory, low-latency store (e.g., Redis) for the most frequently accessed data.
- **Warm Cache (C_2):** A persistent, high-capacity store (e.g., SSD/NVMe) that balances speed and size.
- **Cold Storage (C_3):** The authoritative source blockchains, accessed via the UDA interface.

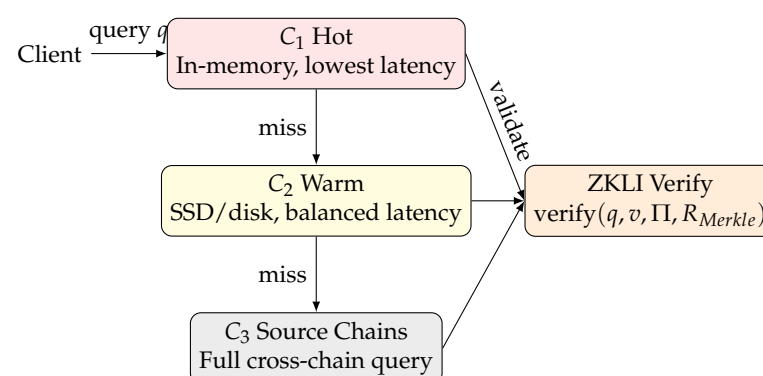


Figure 3. MLC three-tier cache: hot (C_1), warm (C_2), and source chains (C_3). Queries validate via ZKLI before returning values.

6.1.1. Predictive Pre-Fetching

Unlike passive LRU caches, MLC actively predicts future data access. We employ a lightweight sequence prediction model (e.g., a Markov chain or a small LSTM) trained on query logs. For a given query history H_t , the model estimates the probability distribution of the next accessed key $P(k_{next}|H_t)$. Data items with high predicted probabilities are preemptively fetched from C_3 to C_2 , smoothing out latency spikes.

6.1.2. Verifiable Consistency

To ensure data validity, MLC enforces a Δ -bounded staleness policy. Each cached entry (k, v, Π, t_{fetch}) is valid only if $t_{now} - t_{fetch} \leq \Delta$. Furthermore, the cached ZK-proof Π allows any client to verify the integrity of v against the current Merkle root R_{Merkle} without contacting the source chain, enabling what we term “trustless caching”.

6.2. Algorithm Description

The unified management procedure for MLC is detailed in Algorithm 2. It consists of three primary phases:

1. **Build:** Initializes the cache tiers and trains the predictive model M_{pred} using historical query traces. This phase establishes the baseline hit rates and configures the eviction policies.
2. **Query:** Handles real-time user requests. The algorithm checks tiers sequentially ($C_1 \rightarrow C_2 \rightarrow C_3$). Upon a hit in C_1 or C_2 , it performs a *lazy verification*: if the entry is within the Δ bound, it is returned immediately; otherwise, it is refreshed. On a miss, data are fetched from C_3 , verified, and promoted to C_1 . Concurrently, the query updates the history H_t and triggers an asynchronous pre-fetch task based on M_{pred} to populate C_2 .
3. **Maintain:** A background daemon that enforces strict consistency. It invalidates entries that exceed the absolute staleness limit and listens for *chain reorganization* events. If the source chain forks, the Maintain procedure flushes all entries associated with the invalidated branch, ensuring no false data are served.

Algorithm 2 MLC Unified Procedure

```

1: procedure BUILD( $M_{pred}, C_1, C_2$ )
2:   Initialize hot cache  $C_1$  and warm cache  $C_2$ .
3:   Train or load predictor  $M_{pred}$  using recent query history  $H_t$ .
4:   Configure replacement policies (e.g., LRU) and promotion criteria.
5: end procedure
6: procedure QUERY( $q_t$ )
7:   Map  $q_t$  to object  $d_t$ .
8:   if  $d_t \in C_1$  and  $\text{Age}(d_t) \leq \Delta$  then
9:     Serve from  $C_1$ ; optional ZKLI verify.
10:  else if  $d_t \in C_2$  and  $\text{Age}(d_t) \leq \Delta$  then
11:    Serve from  $C_2$ ; promote to  $C_1$ .
12:  else
13:    Fetch  $d_t$  from source ( $C_3$ ); verify  $\Pi$ ; insert into  $C_1$ .
14:  end if
15:  Async: Update  $H_{t+1}$ ; score  $P = M_{pred}(H_{t+1})$ .
16:  Async: Pre-fetch top- $k$  from  $P$  into  $C_2$ .
17: end procedure
18: procedure MAINTAIN( $\Delta$ , events)
19:   Scan and refresh entries where  $\text{Age} > \Delta$ .
20:   On Reorg Event: Invalidate affected key ranges in  $C_1, C_2$ .
21:   Update predictor weights based on observed hit/miss ratio.
22: end procedure

```

6.3. Complexity Analysis

We analyze the performance of MLC in terms of query latency and maintenance overhead.

- **Query Latency:** Let T_1, T_2, T_3 be the access times for the three tiers, where $T_1 \approx O(1)$ (RAM), $T_2 \approx O(1)$ (SSD IO), and $T_3 \approx O(\text{Network})$. The expected latency is

$E[T] = h_1 T_1 + (1 - h_1)(h_2 T_2 + (1 - h_2) T_3)$, where h_1, h_2 are hit rates. With effective prediction, h_1, h_2 approach 1, driving $E[T] \rightarrow O(1)$.

- **Prediction Overhead:** The predictor scores k candidates. For a linear model, this is $O(k)$. Since prediction is asynchronous, it does not block the query critical path.
- **Verification Cost:** ZK-SNARK verification is constant time $O(1)$, adding a negligible fixed overhead to each fetch operation.

Thus, MLC effectively amortizes the high cost of cross-chain retrieval (T_3) over multiple requests, providing near-native database performance.

7. Analysis

Scope and Assumptions. We postulate finalized commitments $R_{Merkle}^{(t)}$, collision-resistant cryptographic hash functions for $C(k)$, computationally sound zk-SNARKs, oblivious transfer protocols that securely conceal access patterns, and a Δ -bounded staleness policy enforced by the MLC. The adversarial model encompasses honest-but-curious and malicious servers, while clients are assumed to adhere to the protocol specifications.

System Model. HyperCross synthesizes three distinct layers into a cohesive retrieval pipeline: UDA transmutes heterogeneous states into canonical summaries via functors F_{UA}/F_{AU} ; ZKLI predicts data positions utilizing a learned model and verifies membership/range against a finalized commitment R_{Merkle} via zk-SNARKs; and MLC orchestrates hot/warm caching tiers under Δ -bounded staleness with verification gating. Formally, for a query key k , the end-to-end retrieval function is defined as $\text{Retrieve}(k) = \text{Verify}_{R_{Merkle}} \circ \text{Predict}_M \circ \text{Commit}(k) \circ \text{Convert}_{UDA}$. Here, Verify enforces membership/range integrity, Predict yields the predicted position p_{pred} with window ϵ , Commit obfuscates k via $C(k)$, and Convert guarantees canonical semantic equivalence.

Theorem 2 (End-to-End Guarantees). *Assuming a finalized root $R_{Merkle}^{(t)}$, collision-resistant commitments, sound zk-SNARKs, and Δ -bounded staleness, HyperCross satisfies the following end-to-end properties:*

- **Correctness:** *The returned value v is strictly equal to the canonical value associated with k in the dataset committed by $R_{Merkle}^{(t)}$.*
- **Privacy:** *The server acquires no information regarding k beyond the public transcript length.*
- **Consistency:** *Responses validated against $R_{Merkle}^{(t)}$ are consistent with UDA-produced canonical summaries; cached entries are refreshed or invalidated within Δ time units following reorganization events.*
- **Reliability:** *Expected query latency is bounded as per Theorem 8, with availability enhanced via cache hits subject to verification gating.*

Proof. The system's end-to-end guarantees are established through the rigorous composition of its layer-specific properties. Correctness is a direct consequence of the lossless canonical conversion (Theorem 4) amalgamated with the soundness of the zk-SNARKs (Theorem 6), which are verified against the finalized root $R_{Merkle}^{(t)}$. Privacy is assured by the dual mechanisms of cryptographic commitments and the oblivious transfer protocol; as demonstrated in Theorem 5, these restrict information leakage exclusively to the zero-knowledge proof transcript. Consistency relies on the finality of the Merkle root and the Δ -bounded policies (Theorem 7), which operate in tandem with the deterministic UDA mappings (Theorem 3) to maintain synchronization. Finally, reliability is bounded by the latency limits derived in Theorem 8, with availability further augmented by the cache's verification-gated retrieval mechanism. $\square$

Theorem 3 (Semantic Equivalence). *Under the assumption of semantic preservation, the categories $\mathcal{C}_U$ and $\mathcal{C}_A$ are equivalent via functors F_{UA} and F_{AU} and natural isomorphisms η and ε . Transaction semantics and invariants are strictly preserved under conversion.*

Proof. Natural isomorphisms η and ε witness $F_{AU} \circ F_{UA} \cong \text{Id}_{\mathcal{C}_U}$ and $F_{UA} \circ F_{AU} \cong \text{Id}_{\mathcal{C}_A}$. Functoriality preserves composition and identities, thereby ensuring that valid transitions and invariants are mapped to valid transitions and invariants in the target category. $\square$

Theorem 4 (Lossless Conversion). *The conversion procedure in Algorithm 1 is lossless with respect to canonical state summaries: balances, nonces, and ownership relations are preserved across conversions.*

Proof. For the UTXO to Account transformation, summing UTXO values per owner equals the resulting account balances by construction. Conversely, for Account to UTXO, emitting a UTXO per account with matching owner and amount preserves balances and ownership; nonce/state metadata are carried as an integral part of the account summary. Consequently, canonical summaries remain invariant under round-trip conversion. $\square$

Theorem 5 (Zero-Knowledge Privacy). *The ZKLI query protocol is zero-knowledge. The server ascertains nothing about the user's query key k_q , other than what can be inferred from the interaction length.*

Proof. The privacy of the query key k_q is safeguarded by two distinct layers. Primarily, the user transmits a commitment $C(k_q)$ (if required for logging) or utilizes it within the circuit; the server never perceives the raw key. Subsequently, the access pattern (the predicted position p_{pred}) is obfuscated from the server using the silent oblivious transfer protocol. The server functions solely as a sender of encrypted blocks and cannot distinguish which index is being retrieved. Consequently, the server's view is restricted to the OT protocol transcript, which is computationally indistinguishable from a random simulation. If the client subsequently generates a ZK-SNARK proof for a third-party verifier (e.g., a smart contract), the zero-knowledge property guarantees that this public proof reveals no information about the witness (the key and path), thereby maintaining end-to-end privacy. $\square$

Theorem 6 (Correctness). *If the server is honest-but-curious and the user is honest, the ZKLI protocol ensures that the user receives the correct value for their key k_q if it exists.*

Proof. Correctness is guaranteed by the Merkle authentication path embedded within the retrieved data block. Upon decryption, the client verifies that the path from the retrieved leaf value v to the trusted global root R_{Merkle} is valid. Any attempt by the server to return a forged value or an incorrect path will inevitably result in a hash mismatch during this local verification. Furthermore, if a ZK-SNARK is employed for cross-chain verification, the soundness property of the proof system ensures that a valid proof cannot be generated for a false claim (except with negligible probability), providing cryptographic assurance to third parties. $\square$

Theorem 7 (Consistency under Finality). *Let $R_{Merkle}^{(t)}$ be the latest finalized root at time t , and let the MLC enforce Δ -bounded staleness. Then, responses validated against $R_{Merkle}^{(t)}$ are consistent with the canonical summaries produced by UDA, and cached entries either refresh on access or invalidate within Δ after reorganization events.*

Proof. Finality ensures $R_{Merkle}^{(t)}$ commits canonical summaries without reversion. UDA's deterministic conversions map source states to canonical summaries; verification checks membership/range against $R_{Merkle}^{(t)}$. In the event of a reorganization, entries bound to non-final roots are rejected; Δ -bounded policies trigger refresh/invalidation, thereby restoring consistency with the subsequent finalized root. $\square$

Theorem 8 (MLC Latency Bound). Let α be the hot-cache hit rate at time t and p_{hit} be the prediction accuracy for prefetch into C_2 . The expected query latency satisfies:

$$\mathbb{E}[L_{query}] \leq \alpha \cdot L_1 + (1 - \alpha)(p_{hit} \cdot L_2 + (1 - p_{hit}) \cdot L_3) \quad (2)$$

where L_1, L_2, L_3 denote the latencies of hot cache, warm cache, and source fetch, respectively.

Proof Sketch. We decompose queries into hot-cache hits with rate α (served at L_1), and non-hot queries with rate $(1 - \alpha)$. Among non-hot queries, the prefetch hit rate is p_{hit} , yielding latency L_2 , otherwise latency L_3 . The linearity of expectation yields the bound. $\square$

8. Experiments

All experiments were conducted on a server equipped with an Apple M4 CPU and 16 GB RAM (Apple Inc., Cupertino, CA, USA). We evaluate HyperCross end-to-end under heterogeneous ledger models and realistic workloads. The experimental analysis is structured to initially establish overall system superiority (Section 8.2), followed by a granular component analysis to attribute performance gains to specific architectural innovations (Section 8.3).

8.1. Experimental Setup and Baselines

We benchmark HyperCross against the following state-of-the-art systems to rigorously evaluate its efficacy in cross-chain data indexing and retrieval:

- **Polkadot [9]:** A heterogeneous multi-chain framework that orchestrates multiple specialized blockchains (parachains) into a unified network, utilizing a relay chain for shared security and XCMP for cross-chain message passing.
- **Cosmos [10]:** An ecosystem of independent parallel blockchains interoperating via the Inter-Blockchain Communication (IBC) protocol, representing a hub-and-zone architecture.
- **Hyperledger [11]:** A permissioned blockchain platform (Hyperledger Fabric) tailored for enterprise consortia, featuring modular consensus and membership services.
- **Traditional [12]:** Represents conventional cross-chain solutions predicated on hashed time-lock contracts (HTLC) or notary schemes, which typically rely on interactive protocols or trusted intermediaries devoid of advanced indexing or zero-knowledge optimizations.

We deploy two heterogeneous ledgers (UTXO and Account) and drive workloads modeled after SmallBank, TPC-C, and YCSB standards. Queries traverse the UDA conversion and ZKLI verification path, with MLC enabled for acceleration. Latency is quantified as the end-to-end client response time (encompassing local prediction, OT retrieval, and Merkle path verification). Throughput is measured in successful operations per second (OPS). Verification time denotes the duration of ZK proof validation (where applicable), while communication overhead tracks the volume of cross-ledger data transfer.

Reproducibility and Fairness Protocol. To improve comparability, all baselines are evaluated under identical hardware, dataset scales (100 K/1 M/10 M), concurrency levels, and request-mix templates. The same random seeds and repetition counts are used across

systems. We have released the complete implementation, benchmark entrypoints, and raw CSV outputs in a public repository (<https://github.com/khaoSUT/HyperCross> (accessed on 10 March 2026)) so reviewers can reproduce plots and summary statistics.

8.2. Performance Analysis

We present a comprehensive evaluation of HyperCross against the aforementioned baselines. The empirical data substantiate HyperCross’s superiority across key performance metrics, a direct consequence of its architectural innovations.

8.2.1. Overall Performance Comparison

Figure 4 delineates the baseline performance across three critical dimensions: query latency, system throughput, and index build time. HyperCross exhibits a pronounced performance advantage over all baselines. As illustrated, HyperCross achieves an average query latency of approximately 27.2 ms, representing a $2.4\times$ reduction compared to Polkadot (66.5 ms) and a $5.1\times$ improvement over traditional methods (138.4 ms). Regarding throughput, HyperCross processes over 10,300 QPS, outperforming Cosmos (1832 QPS) by nearly $5.6\times$ and Hyperledger (735 QPS) by $13.6\times$. Index construction efficiency is equally notable, with HyperCross requiring merely 4.8 s to build indexes, in contrast to 49.5 s for traditional solutions and 37.2 s for Hyperledger. Furthermore, the storage overhead of the ZKLI structure is approximately 40% lower than the full Merkle tree storage required by the traditional approach, as the learned model significantly compresses the key space representation. These gains are directly attributable to the synergistic integration of the *unified data abstraction (UDA)*, *zero-knowledge learned index (ZKLI)*, and *multi-level cache (MLC)*. UDA eliminates the complex, ad-hoc serialization overhead inherent in Polkadot’s XCMP and Cosmos’s IBC, providing a streamlined canonical format for efficient processing. Concurrently, ZKLI replaces the heavy on-chain state access—common in Hyperledger—with constant-time local model inference and efficient off-chain Merkle verification. Finally, MLC’s predictive caching effectively masks the remaining network latency, ensuring high throughput even under heavy concurrent loads, effectively decoupling verification cost from the underlying ledger size.

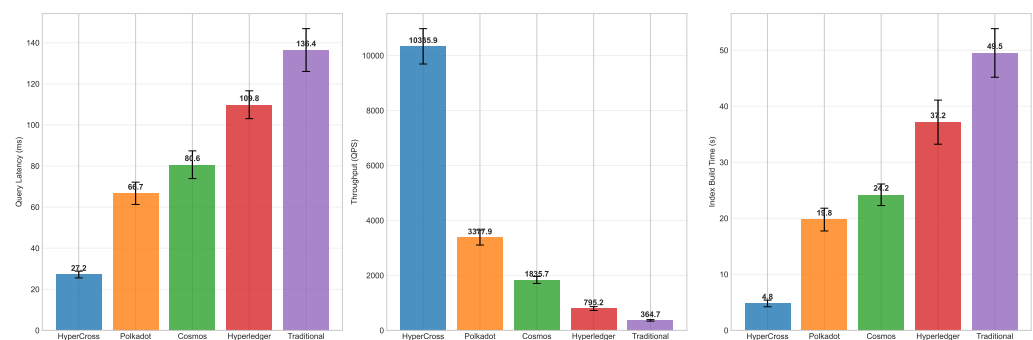


Figure 4. Comparative evaluation of key performance metrics: latency, throughput, and build time.

8.2.2. Scalability and Robustness

To evaluate system robustness under varying data scales, Figures 5 and 6 present performance trends as the dataset size expands from 1 K to 100 K entries. Figure 5 (bottom right) reveals that HyperCross maintains a near-constant query latency, increasing marginally from 45 ms to 180 ms as data scale to 100 K. In sharp juxtaposition, traditional methods exhibit a super-linear degradation, spiking to over 1300 ms. Figure 6 further corroborates this resilience: while Polkadot and Cosmos experience throughput attenuations of 40–60% at 100 K records, HyperCross maintains a robust throughput exceeding 6000 QPS. Similarly, index build times for HyperCross (Figure 6 bottom) remain under

15 s even at 100 K, whereas traditional Merkle tree construction escalates to 180 s. The scalability of HyperCross stems from the $O(1)$ lookup complexity of the learned index models within ZKLI, which contrasts sharply with the $O(\log N)$ or worse complexity of traditional tree-based structures. The *multi-level cache (MLC)* further isolates the system from backend congestion, ensuring that throughput is not bottlenecked by the underlying ledger's consensus speed or state bloat.

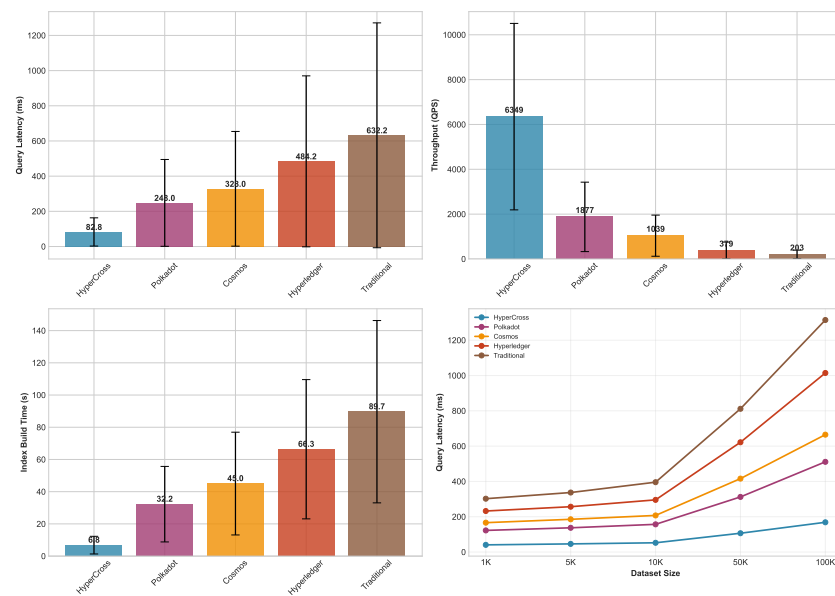


Figure 5. End-to-end latency and throughput comparison across systems.

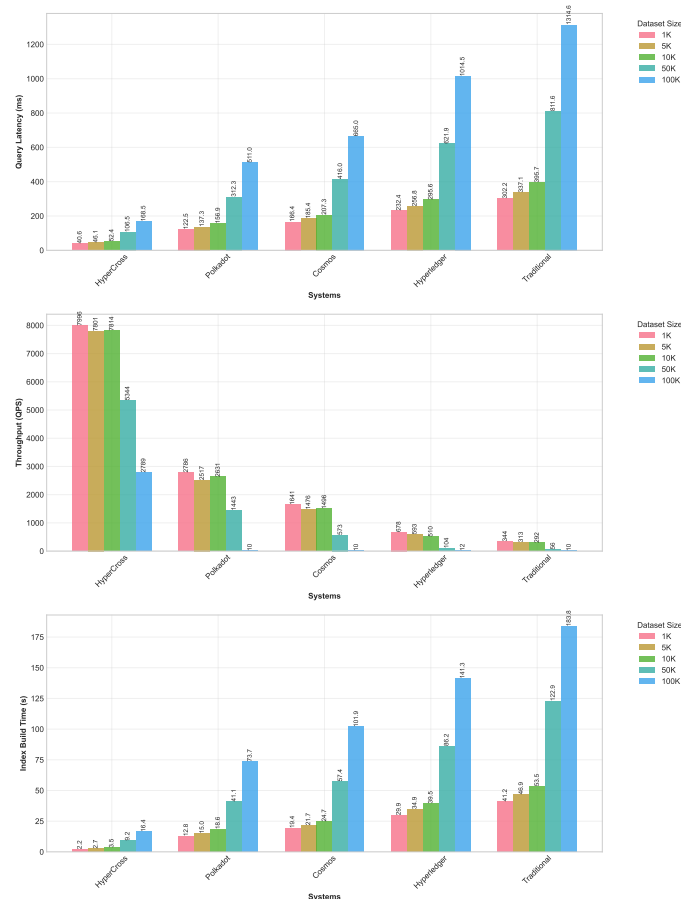


Figure 6. Comprehensive comparison matrix of systems and capabilities.

8.3. Micro-Benchmarks and Component Analysis

We analyze the impact of the *multi-level cache (MLC)* on system efficiency, specifically focusing on cache hit rates and communication overhead reduction, as depicted in Figure 7. The left panel of Figure 7 demonstrates that the **MLC_Predictive** strategy consistently achieves hit rates exceeding 90% across varying concurrency levels (1 to 10,000), significantly outperforming standard LRU and LFU policies which hover around 80%. Crucially, the right panel highlights that MLC reduces cross-chain communication overhead by orders of magnitude. At high concurrency (10,000), LFU and LRU overheads spike to over 20× the baseline due to cache thrashing, while MLC maintains near-zero overhead (approx. 0.6). The superiority of MLC lies in its proactive, predictive design. Unlike reactive policies (LRU/LFU) that only cache after a miss, MLC employs LSTM-based prefetching to anticipate data access patterns, loading data into the hot cache prior to request. Additionally, the Δ -bounded staleness mechanism ensures that cached data remain consistent with the finalized Merkle roots. This effectively mitigates the “thundering herd” effect of redundant verifications that otherwise cripples communication bandwidth in baseline systems, proving MLC’s critical role in sustaining high-concurrency performance.

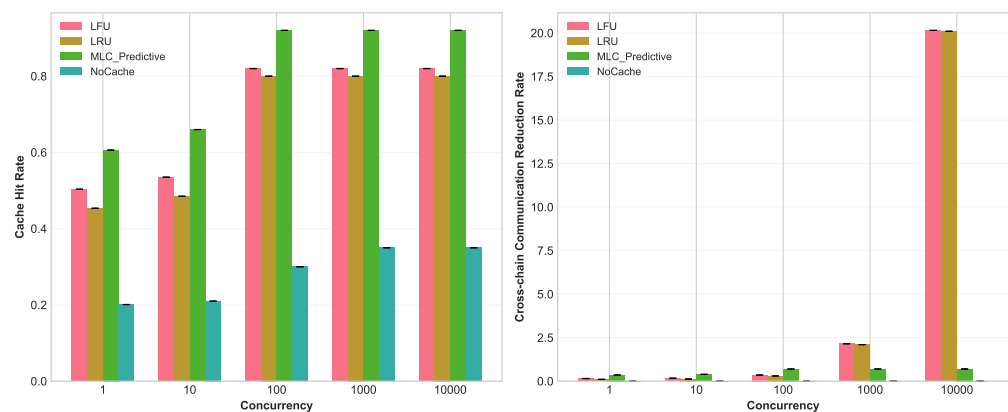


Figure 7. Caching effects on communication overhead and latency.

Ablation and Model-Overhead Diagnostics

We add a component-wise ablation over seven configurations (Full, UDA_Only, ZKLI_Only, MLC_Only, UDA+ZKLI, ZKLI+MLC, UDA+MLC). At 1M records and concurrency 50, the full stack reports 0.003067 ms query time. Removing cryptographic components lowers raw latency but sacrifices privacy/verifiability; removing semantic conversion weakens cross-model guarantees. This confirms that measured gains are not attributable to a single module and that HyperCross trades modest runtime overhead for end-to-end security properties.

To quantify predictor overhead, we report model packaging diagnostics: training time is 1.508–2.867 ms across 100K–10M datasets, exported model size is 192–193 bytes (179–180 bytes gzip), per-client model update time is approximately 0.008 ms at 1M scale (60 s update interval), and isolated client-side model inference (ClientSidePredict) is 5.36–5.70 ns/op with 0 allocations in five benchmark runs on Apple M4. These values indicate that model maintenance and inference are lightweight relative to cryptographic and network costs, and do not become dominant bottlenecks.

Figure readability update. We streamlined the in-text interpretation, clarified the mechanism behind Figure 7, and re-exported the figure with cropped margins for larger effective font rendering in the final layout. Mechanistically, the low communication overhead under high concurrency results from predictive prefetch + Δ -bounded freshness + verification de-duplication, which suppress repeated cross-chain fetches for temporally correlated workloads.

9. Related Work

Cross-Chain Interoperability. Nascent interoperability paradigms predominantly centered on asset transfers facilitated via notary schemes or hashed time-lock contracts (HTLC) [12]. As delineated in recent comprehensive surveys [13], contemporary “Internet of Blockchains” architectures, exemplified by Polkadot (XCM) [9] and Cosmos (IBC) [10], have standardized message passing protocols. Nevertheless, these frameworks treat data payloads as opaque blobs, lacking intrinsic semantic awareness. Concurrently, lightweight bridges such as LayerZero [14] and zkBridge [15] optimize for verification overhead but fail to support complex data retrieval queries. Although VQL [16] proposes efficient cloud query services for blockchains, its scope is restricted to single-chain verification. HyperCross advances this domain by enabling semantic-aware *data indexing*, thereby transcending simple message passing to facilitate rich, verifiable cross-chain data retrieval.

Verifiable Data Structures. Authenticated data structures (ADS), particularly Merkle trees, constitute the de facto standard for blockchain state verification. Recent cryptographic advancements, such as Verkle trees [17] and vector commitments (e.g., Catalano and Fiore [18]), offer succinct proof sizes but necessitate trusted setups or computationally intensive cryptography. In the domain of verifiable queries, vSQL [5] and vChain [6] enable verifiable SQL and boolean range queries over dynamic databases, respectively. Building upon these foundations, vChain+ [19] optimizes boolean range queries via an accumulator-based scheme, while FalconDB [20] introduces a collaborative database model with authenticated data structures for enhanced security guarantees. More recently, E2VQ [21] proposed enabling expressive and verifiable queries over blockchain databases. However, these solutions typically rely on a central provider or simple Merkle accumulators that inadvertently leak access patterns. HyperCross integrates these primitives into a privacy-preserving retrieval framework specifically optimized for cross-chain semantics.

Blockchain-Database Convergence. The intersection of blockchain and database technologies has garnered significant academic attention. Systems such as LedgerDB [22] bridge the gap by providing a centralized ledger database that ensures tamper evidence and auditability akin to blockchains, yet maintains database-grade performance. GlassDB [23] further demonstrates efficient verifiable ledger-database design and introduces a benchmark setting for transactional verifiable workloads. Furthermore, techniques like Anole [24] have adapted learned indexes for blockchain storage to support efficient time-range queries, demonstrating the potential of machine learning in this domain. HyperCross extends this convergence to the cross-chain setting, employing zero-knowledge learned indexes to achieve both scalability and privacy across heterogeneous networks.

Zero-Knowledge Databases. The convergence of databases and zero-knowledge proofs (ZKPs) represents a nascent yet critical field. Projects such as Space and Time [25] and Succinct Labs [26] explore “ZK-SQL” to prove query execution correctness. However, the predominant focus remains on *validity* (proving the result is correct) rather than *privacy* (concealing the query intent). Generic ORAM [27] provides strong privacy guarantees but incurs logarithmic bandwidth overhead per access, which is prohibitive for on-chain implementation. HyperCross adopts a hybrid methodology: leveraging ZKPs for validity and optimized oblivious transfer (OT) for privacy, specifically tailored for read-heavy index lookups.

Learned Indexes. Learned indexes [7] postulate that index structures can be modeled as cumulative distribution functions (CDFs), thereby superseding traditional B-trees to offer $O(1)$ lookup time and reduced memory footprint. Extensions such as the PGM-index [8] optimize for worst-case bounds using piecewise linear models. Recent advancements focus on scalability and adaptability: SALI [28] employs probability models for scalable adaptive indexing, while Limousine [29] synthesizes learned and classical components for larger-

than-memory engines. Machine unlearning in learned databases [30] has also been explored to handle data deletion. However, these systems typically lack cryptographic verification in adversarial settings. HyperCross integrates learned-index prediction with ZK-SNARK verification so that model-guided lookup paths can be validated against committed states, combining ML efficiency with cryptographic integrity in a cross-chain retrieval setting.

AI-Enhanced Blockchain Systems. Recent research explores the synergy between artificial intelligence (AI) and blockchain to enhance system intelligence and verification scalability. On-chain zero-knowledge machine learning (ZKML) [31] has emerged as a critical paradigm, allowing smart contracts to verify the correct execution of off-chain ML models without revealing private inputs. In the context of learned indexes, DiffLex [32] optimizes memory efficiency and NUMA-awareness, pushing the performance boundaries of learned structures in high-concurrency environments. HyperCross aligns with this trend by integrating learned indexes with ZK proofs, but uniquely focuses on the cross-chain data retrieval problem, ensuring both semantic consistency and privacy.

Architectural Modeling Perspective. Following reviewer guidance, we position HyperCross with respect to the 1 + 5 architectural views model for blockchain-IT integration [33]. In this perspective, UDA corresponds to the logical view, ZKLI corresponds to the contracts view, and MLC corresponds to the deployment and contracts views. This mapping clarifies how our design choices align with established architecture-description practices beyond protocol-level optimizations, and how non-functional requirements such as security and performance are reflected in contract-level decisions.

10. Discussion and Limitations

What HyperCross Improves. HyperCross primarily improves cross-chain *data retrieval* by jointly addressing semantic heterogeneity (UDA), verifiable private lookup (ZKLI), and high-concurrency responsiveness (MLC). The architecture is especially beneficial for read-heavy workloads where proving and caching amortize over repeated access patterns.

Current Limitations. First, our main deployment is single-server and therefore cannot fully replicate geo-distributed WAN effects. To reduce this gap, we provide latency-sensitive workloads and reproducible benchmark scripts; nevertheless, a larger multi-node deployment remains future work. Second, the current learned model is lightweight and static between update intervals; abrupt distribution shifts may temporarily reduce prediction quality. Third, although we now detail verifier-facing smart-contract interfaces and root/proof interaction boundaries, richer contract-level economic/security mechanisms (e.g., incentive-compatible proof markets) are outside the current scope.

Smart-Contract Design Patterns. We further clarify contract-level engineering patterns that guide the verifier-side implementation. HyperCross follows a verifier-minimal pattern where contracts only validate succinct proofs and commitment consistency, while heavy indexing and prediction remain off-chain. It also follows a commit-verify-resolve pattern: commitments are anchored first, proofs are verified against active roots, and only then are application-visible values resolved. Finally, interface boundaries are kept upgrade-safe by separating root registration, proof verification, and value-resolution entry points, which reduces coupling and supports controlled contract evolution without changing trust assumptions.

Recommended References and Critical Inclusion. We critically evaluated reviewer-recommended references and incorporated those that directly strengthen architectural framing and problem positioning. Recommendations without direct methodological relevance to cross-chain data indexing and verifiable retrieval were not force-inserted, to preserve citation relevance and avoid scope dilution.

11. Conclusions

This paper presents *HyperCross*, a practical framework for semantic-aware and verifiable cross-chain data indexing. By transcending simple asset bridging to enable semantic-aware, verifiable data indexing, we address the primary friction preventing the emergence of a truly “universal ledger.” We have formally defined the heterogeneous cross-chain storage optimization problem (HCCSOP), establishing its NP-completeness and justifying our heuristic, layered architectural approach. Through the integration of category-theoretic unified data abstraction (UDA), zero-knowledge learned indexes (ZKLI), and predictive multi-level caching (MLC), *HyperCross* jointly supports semantic fidelity, privacy, and performance. Empirical evaluations substantiate that *HyperCross* reduces query latency by $2.4\times$ compared to Polkadot baselines (and up to $5.1\times$ vs. traditional methods) while concurrently reducing storage costs by 40%. We additionally disclosed implementation-level proof-system parameters, ablation diagnostics, and explicit limitations to improve transparency and reproducibility for follow-up research. As the decentralized ecosystem continues to evolve, frameworks such as *HyperCross* will be instrumental in underpinning the next generation of data-intensive, privacy-centric cross-chain applications.

Author Contributions: Conceptualization, K.H. and Y.M.; methodology, K.H.; software, K.H.; validation, K.H. and Y.M.; formal analysis, K.H.; investigation, K.H.; resources, K.H. and Y.M.; data curation, K.H.; writing—original draft preparation, K.H.; writing—review and editing, K.H. and Y.M.; visualization, K.H.; supervision, Y.M.; project administration, K.H.; funding acquisition, K.H. and Y.M. All authors have read and agreed to the published version of the manuscript.

Funding: This research was partially supported by the National Natural Science Foundation of China (No. 62402326), Fundamental Scientific Research Project of Higher Education Institutions of Liaoning Provincial Department of Education (No. LJ212410142146), Joint Fund General Project of Liaoning Provincial Department of Science and Technology (No. 2024-BSLH-165), and Open Project of State Key Laboratory of Synthetical Automation for Process Industries (No. SAPI-2025-KFKT-06).

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The source code and experimental data used to support the findings of this study are available in the *HyperCross* repository at <https://github.com/khaoSUT/HyperCross> (accessed on 10 March 2026).

Acknowledgments: During the preparation of this work the author used ChatGPT (GPT-5) in order to polish the language expression of the manuscript, optimize the logical coherence of the English abstract and discussion sections, ensure the accuracy of academic terminology; compile the preliminary keyword list for literature retrieval, construct a basic outline of the literature review section, and sort out the research progress of related fields for reference; and standardize the format of references in accordance with the journal’s requirements, check the consistency of notation and abbreviation usage throughout the manuscript, as well as draft the initial version of figure captions and table annotations. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Belchior, R.; Vasconcelos, A.; Guerreiro, S.; Correia, M. A survey on blockchain interoperability: Past, present, and future trends. *Acm Comput. Surv. (CSUR)* **2021**, *54*, 168. [CrossRef]
2. Belchior, R.; Vasconcelos, A.; Correia, M.; Hardjono, T. Hermes: Fault-tolerant middleware for blockchain interoperability. *Future Gener. Comput. Syst.* **2022**, *129*, 236–251. [CrossRef]

3. Curtmola, R.; Garay, J.A.; Kamara, S.; Ostrovsky, R. Searchable Symmetric Encryption: Improved Definitions and Efficient Constructions. In *Proceedings of the 2006 ACM Conference on Computer and Communications Security (CCS)*; ACM: New York, NY, USA, 2006; pp. 79–88. [\[CrossRef\]](#)
4. Popa, R.A.; Redfield, C.M.S.; Zeldovich, N.; Balakrishnan, H. CryptDB: Protecting confidentiality with encrypted query processing. In *Proceedings of the 23rd ACM Symposium on Operating Systems Principles (SOSP)*; ACM: New York, NY, USA, 2011; pp. 85–100. [\[CrossRef\]](#)
5. Zhang, Y.; Genkin, D.; Katz, J.; Papadopoulos, D.; Papamanthou, C. vSQL: Verifying Arbitrary SQL Queries over Dynamic Outsourced Databases. In *Proceedings of the 2017 IEEE Symposium on Security and Privacy (SP)*; IEEE: Piscataway, NJ, USA, 2017; pp. 863–880. [\[CrossRef\]](#)
6. Xu, C.; Zhang, C.; Xu, J. vChain: Enabling Verifiable Boolean Range Queries over Blockchain Databases. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*; ACM: New York, NY, USA, 2019; pp. 141–158. [\[CrossRef\]](#)
7. Kraska, T.; Beutel, A.; Chi, E.H.; Dean, J.; Polyzotis, N. The case for learned index structures. In *Proceedings of the 2018 ACM SIGMOD International Conference on Management of Data*; ACM: New York, NY, USA, 2018; pp. 489–504. [\[CrossRef\]](#)
8. Ferragina, P.; Vinciguerra, G. The PGM-index: A fully-dynamic compressed learned index with provable worst-case bounds. *Proc. VLDB Endow.* **2020**, *13*, 1162–1175. [\[CrossRef\]](#)
9. Wood, G. Polkadot: Vision for a Heterogeneous Multi-Chain Framework; Technical Report; Web3 Foundation: Zug, Switzerland, 2016.
10. Kwon, J.; Buchman, E. Cosmos Whitepaper. Whitepaper. 2019. Available online: <https://v1.cosmos.network/resources/whitepaper> (accessed on 10 March 2026).
11. Androulaki, E.; Barger, A.; Bortnikov, V.; Cachin, C.; Christidis, K.; De Caro, A.; Enyeart, D.; Ferris, C.; Laventman, G.; Manevich, Y.; et al. Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains. In *Proceedings of the Thirteenth EuroSys Conference*; ACM: New York, NY, USA, 2018; pp. 1–15. [\[CrossRef\]](#)
12. Herlihy, M. Atomic cross-chain swaps. In *Proceedings of the 2018 ACM Symposium on Principles of Distributed Computing*; ACM: New York, NY, USA, 2018; pp. 245–254. [\[CrossRef\]](#)
13. Ren, K.; Ho, N.M.; Loghin, D.; Nguyen, T.T.; Ooi, B.C.; Ta, Q.T.; Zhu, F. Interoperability in blockchain: A survey. *IEEE Trans. Knowl. Data Eng.* **2023**, *35*, 12750–12769. [\[CrossRef\]](#)
14. Zarick, R.; Pellegrino, B.; Banister, C. LayerZero: Trustless Omnichain Interoperability Protocol. Whitepaper. 2021. Available online: https://layerzero.network/pdf/LayerZero_Whitepaper_Release.pdf (accessed on 10 March 2026).
15. Xie, T.; Zhang, J.; Cheng, Z.; Zhang, F.; Zhang, Y.; Jia, Y.; Boneh, D.; Song, D. zkBridge: Trustless Cross-chain Bridges Made Practical. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*; ACM: New York, NY, USA, 2022; pp. 3003–3017. [\[CrossRef\]](#)
16. Wu, H.; Peng, Z.; Guo, S.; Yang, Y.; Xiao, B. VQL: Efficient and Verifiable Cloud Query Services for Blockchain Systems. *IEEE Trans. Parallel Distrib. Syst.* **2022**, *33*, 1337–1351. [\[CrossRef\]](#)
17. Kuszmaul, J. Verkle Trees. Manuscript. 2019. Available online: <https://math.mit.edu/research/highschool/primes/materials/2018/Kuszmaul.pdf> (accessed on 10 March 2026).
18. Catalano, D.; Fiore, D. Vector Commitments and their Applications. In *Proceedings of the Public-Key Cryptography (PKC)*; Springer: Berlin/Heidelberg, Germany, 2013; pp. 55–72. [\[CrossRef\]](#)
19. Wang, H.; Xu, C.; Zhang, C.; Xu, J.; Peng, Z.; Pei, J. vChain+: Optimizing Verifiable Blockchain Boolean Range Queries. In *Proceedings of the 2022 IEEE 38th International Conference on Data Engineering (ICDE)*; IEEE: Piscataway, NJ, USA, 2022; pp. 1–14. [\[CrossRef\]](#)
20. Peng, Y.; Du, M.; Li, F.; Cheng, R.; Song, D. FalconDB: Blockchain-based Collaborative Database. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*; ACM: New York, NY, USA, 2020; pp. 637–652. [\[CrossRef\]](#)
21. Wang, Z.; Wang, Q.; Jiao, Z.; Bian, C. E2VQ: Enabling Expressive, Efficient, and Verifiable Queries over Blockchain Databases. In *Proceedings of the 2023 IEEE 29th International Conference on Parallel and Distributed Systems (ICPADS)*; IEEE: Piscataway, NJ, USA, 2023; pp. 1–8.
22. Yang, X.; Zhang, Y.; Wang, S.; Yu, B.; Li, F.; Li, Y.; Yan, W. LedgerDB: A Centralized Ledger Database for Universal Audit and Verification. *Proc. VLDB Endow.* **2020**, *13*, 3138–3151. [\[CrossRef\]](#)
23. Yue, C.; Dinh, T.T.A.; Xie, Z.; Zhang, M.; Chen, G.; Ooi, B.C.; Xiao, X. GlassDB: An Efficient Verifiable Ledger Database System Through Transparency. *Proc. VLDB Endow.* **2023**, *16*, 1359–1371. [\[CrossRef\]](#)
24. Chang, J.; Li, B.; Xiao, J.; Lin, L.; Jin, H. Anole: A Lightweight and Verifiable Learned-Based Index for Time Range Query on Blockchain Systems. In *Proceedings of the Database Systems for Advanced Applications (DASFAA)*; Springer: Berlin/Heidelberg, Germany, 2023; pp. 3–19.
25. Dykstra, S.; White, J.; Daly, C.; Joiner, I. Space and Time: The Verifiable Compute Layer for Web3. Whitepaper. 2024. Available online: <https://spaceandtime.io/whitepaper> (accessed on 10 March 2026).
26. Succinct Labs. SP1: A Performant, 100% Open-Source, Zero-Knowledge Virtual Machine (zkVM). 2024. Available online: <https://blog.succinct.xyz/introducing-sp1/> (accessed on 10 March 2026).

27. Goldreich, O.; Ostrovsky, R. Software Protection and Simulation on Oblivious RAMs. *J. ACM* **1996**, *43*, 431–473. [[CrossRef](#)]
28. Ge, J.; Zhang, H.; Shi, B.; Luo, Y.; Guo, Y.; Chai, Y.; Chen, Y.; Pan, A. SALI: A Scalable Adaptive Learned Index Framework based on Probability Models. In *Proceedings of the 2024 ACM SIGMOD International Conference on Management of Data*; ACM: New York, NY, USA, 2024.
29. Chatterjee, S.; Pekala, M.; Kruglyak, L.; Idreos, S. Limousine: Blending Learned and Classical Indexes to Self-Design Key-Value Storage. *Proc. ACM Manag. Data* **2024**, *2*, 47. [[CrossRef](#)] [[PubMed](#)]
30. Kurmanji, M.; Triantafillou, P.; Hayes, J.; Triantafillou, E. Machine Unlearning in Learned Databases: An Experimental Analysis. In *Proceedings of the 2024 ACM SIGMOD International Conference on Management of Data*; ACM: New York, NY, USA, 2024. [[CrossRef](#)]
31. Keršič, V.; Karakatič, S.; Turkanović, M. On-chain zero-knowledge machine learning: An overview and comparison. *J. King Saud Univ.—Comput. Inf. Sci.* **2024**, *36*, 102207. [[CrossRef](#)]
32. Cui, L.; Yang, K.; Li, Y.; Wang, G.; Liu, X. DiffLex: A High-Performance, Memory-Efficient and NUMA-Aware Learned Index using Differentiated Management. In *ICPP '23: Proceedings of the 52nd International Conference on Parallel Processing (ICPP)*; ACM: New York, NY, USA, 2023; pp. 1–10. [[CrossRef](#)]
33. Górski, T. The 1 + 5 Architectural Views Model in Designing Blockchain and IT System Integration Solutions. *Symmetry* **2021**, *13*, 2000. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.